

Format Conversion (Feature H) — Build, Testing & Font Fixes

DOCX/XLSX → PDF via LibreOffice headless — 2026-07-24

Contents

- 1. Context — the findings doc, and one architecture change from it
- 2. What was built
- 3. The findings doc's two fixes — verified, both confirmed necessary
- 4. Gap found beyond the findings doc: page-width column cropping
- 5. Font-substitution issues found through real testing (not in the findings doc at all)
- 6. Full test summary
- 7. Commands reference

1. Context — the findings doc, and one architecture change from it

Implementation started from docs/mizan_format_conversion_findings.md — a pre-existing investigation that had already run `soffice --headless` against real test files and identified two real gotchas (concurrency, XLSX truncation) before any code was written. Scope confirmed from that doc: **DOCX → PDF and XLSX → PDF only**. PDF → DOCX is explicitly out of scope — LibreOffice imports a PDF as a Draw document (positioned shapes, not flowing text), which has no Writer-format export filter at all. That's a hard failure, not a quality problem, and isn't attempted anywhere in this implementation.

ONE DEVIATION FROM THE DOC, CONFIRMED WITH THE USER BEFORE BUILDING

The doc recommended wrapping the core function as an MCP tool inside a shared "document-tools MCP server," deployed on GCP infra separate from the FastAPI gateway. Neither exists anywhere in this repo — confirmed by searching the whole codebase; the only mention of "MCP" anywhere is a single aspirational comment in a different, also-unimplemented feature. Built instead as a plain function wired into a dedicated endpoint in the same `app/main.py` that already hosts every other real feature (RAG, Translate) — same shape as Translate, just without any Modal model call, since this is local subprocess work, not a hosted model.

2. What was built

File	Purpose
agents/doc_format_conversion/config.py	<code>SUPPORTED_SOURCE_EXTENSIONS = {".docx", ".xlsx"}</code> , <code>CONVERT_TIMEOUT_SECONDS = 120</code> , <code>SOFFICE_BINARY</code> (env-overrideable).
agents/doc_format_conversion/convert.py	<code>convert_to_pdf(input_path, source_extension)</code> — the core subprocess wrapper, plus <code>_autofit_xlsx_columns()</code> for

	the XLSX-specific fix. Framework-free, directly unit-testable, exactly as the findings doc's architecture section asked for.
app/main.py	<code>POST /api/convert</code> replaced from a mock (upload accepted but ignored, fake <code>download_url</code> returned) with a real, synchronous, JWT-protected endpoint that returns the converted PDF directly as the response body (<code>FileResponse</code>) rather than a download-URL — there's no file-storage/CDN layer in this repo to generate a real URL against, and a direct response is simpler for v1. Deliberately a plain <code>def</code> , not <code>async def</code> — the work is blocking subprocess/file I/O, and FastAPI runs sync route functions in a thread pool instead of stalling the event loop for every other in-flight request.
services/chatbot/Dockerfile	<code>libreoffice-writer</code> + <code>libreoffice-calc</code> added (confirmed these are the only components actually invoked — <code>writer_pdf_Export</code> / <code>calc_pdf_Export</code> filters — the full <code>libreoffice</code> meta-package pulling in Impress/Draw/Base is unnecessary). Plus three font packages — see §5.
requirements.txt	<code>openpyxl</code> added (needed for the XLSX column auto-fit step).

3. The findings doc's two fixes — verified, both confirmed necessary

Concurrency: unique LibreOffice profile per call

The findings doc reported 1 of 3 concurrent conversions failing silently (return code 1, empty stdout/stderr, no output file, no error message) due to LibreOffice headless instances sharing a default profile lock. Implemented exactly as specified — a fresh `tempfile.mkdtemp()` profile directory per call, passed via `-env:UserInstallation=file://...` , cleaned up in a `finally` block. Verified with 5 simultaneous requests through the real live API:

```
request 0: status=200 size=33147 content-type=application/pdf -> OK
request 1: status=200 size=33147 content-type=application/pdf -> OK
request 2: status=200 size=33147 content-type=application/pdf -> OK
request 3: status=200 size=33147 content-type=application/pdf -> OK
request 4: status=200 size=33147 content-type=application/pdf -> OK
```

All 5 succeeded with identical output size — no lock contention, no silent failures.

XLSX: auto-fit column widths before conversion

The findings doc found long cell text silently truncated in PDF export ("Al Rajhi Trading Co." → "Al Rajhi Tr") because Excel/Calc's on-screen cell-overflow display trick doesn't survive PDF export — every cell clips strictly to its own column width there. Fixed by widening each column to its longest value (via `openpyxl` , on a temp copy — never the caller's original upload) before conversion. Confirmed this alone fixes character-level clipping.

4. Gap found beyond the findings doc: page-width column cropping

FOUND TESTING WITH DATA WIDER THAN THE DOC'S OWN SAMPLE

The findings doc's own test data (a narrow, short-name table) didn't hit this — but testing with a 4-column table including a 45-character vendor name ("International Electronics Distribution Est.") revealed that auto-fitting columns to their content can push the *total table width* past the printable page width (A4 portrait = 595pt). LibreOffice doesn't wrap or shrink to fit in that case — it silently crops the rightmost columns off the page entirely. Confirmed via direct output inspection:

```
Auto-fit only (before this fix):
Vendor Amount (SAR)
Al Rajhi Trading Co. 1000
Saudi Steel Manufacturing Company 5000
International Electronics Distribution Est. 2500

-- "VAT (15%)" and "Total" columns entirely missing, not just clipped --
```

FIX

Added print-scaling alongside the column auto-fit, in the same `_autofit_xlsx_columns()` function:

```
ws.page_setup.fitToWidth = 1
ws.page_setup.fitToHeight = 0
ws.sheet_properties.pageSetUpPr.fitToPage = True
```

Scales the printed sheet to fit one page wide (however many pages tall it needs), on top of the existing column-width fix. Re-tested with the same 4-column, long-vendor-name file — all four columns and every value now present:

```
Vendor Amount (SAR) VAT (15%) Total
Al Rajhi Trading Co. 1000 150 1150
Saudi Steel Manufacturing Company 5000 750 5750
International Electronics Distribution Est. 2500 375 2875
```

5. Font-substitution issues found through real testing (not in the findings doc at all)

Neither of these was in the original findings doc — both surfaced only once real documents were actually visually inspected by a human, which is exactly the kind of gap automated text-extraction checks can't catch (extracted text was correct in both cases; the visual typography was wrong).

Latin text: uneven word spacing

ROOT CAUSE, CONFIRMED VIA FC-MATCH

The container (based on `python:3.11-slim`) shipped with only 9 fonts — the DejaVu family plus OpenSymbol. No Calibri (Word's default font). Confirmed:

```
$ fc-match Calibri
DejaVuSans.ttf: "DejaVu Sans" "Book"
```

DejaVu Sans has different character widths/kerning than Calibri, so any DOCX authored in Word's default font (nearly all of them) reflows with visibly uneven spacing after conversion.

FIX

Added the standard metric-compatible substitute packages to the Dockerfile:

```
fonts-crosextra-carlito    # Calibri substitute
fonts-crosextra-caladea   # Cambria substitute
fonts-liberation2         # Arial / Times New Roman / Courier New substitutes
```

Confirmed after rebuild: `fc-match Calibri` → `Carlito`. User re-tested their original document visually and confirmed the spacing issue is resolved.

Arabic text: same category of issue, given Mizan.ai is Arabic-first

PROACTIVELY CHECKED, NOT REPORTED BY THE USER — SAME ROOT CAUSE PATTERN

Real Word documents commonly use "Traditional Arabic" or "Simplified Arabic" — neither installed. Confirmed:

```
$ fc-match "Traditional Arabic"
DejaVuSans.ttf: "DejaVu Sans" "Book"
```

DejaVu Sans does have Arabic glyph coverage (confirmed via `fc-list :lang=ar`), so Arabic text stays readable and isn't corrupted — but it's a different, non-Arabic-specialized typeface than the source document was authored in.

FIX (WITH ONE WRONG TURN ALONG THE WAY, CORRECTED)

There's no metric-compatible open clone of Microsoft's Arabic fonts the way Carlito is for Calibri — they're proprietary with no open equivalent. First attempt, `fonts-noto-naskh-arabic`, doesn't exist as a standalone package on this Debian release — the build failed cleanly with `E: Unable to locate package` (the running container was untouched; a failed build never replaces it). Checked available options directly via `apt-cache search / policy` against the same base image before trying again, landing on `fonts-kacst-one` — a real, purpose-built Arabic typeface package.

One nuance found while verifying: even after installing KacstOne, standalone `fc-match "Traditional Arabic"` *still* reported DejaVu Sans — because the Calibri→Carlito swap worked via LibreOffice's own hard-coded substitution table (they were built as an official matched pair), which has no equivalent entry for KacstOne. `fc-match` is a generic fontconfig probe, not necessarily what LibreOffice's actual document-rendering engine selects — the two can differ. Flagged this uncertainty explicitly rather than asserting either way, since it can't be resolved without visual inspection. User converted a real Arabic test document and confirmed visually it now renders correctly.

6. Full test summary

Test	Result
DOCX → PDF (heading, paragraphs, bulleted list)	✓ All content correct, real downloadable PDF, correct headers
XLSX → PDF, long cell text, default column widths	✓ No character-level truncation (auto-fit)
XLSX → PDF, 4 wide columns incl. 45-char cell	✓ No column cropping (fitToWidth, added this session)
5x concurrent DOCX conversions	✓ All succeeded, identical correct output
PDF as input (out of scope)	✓ Clean 422 rejection, not silently attempted
Latin text spacing (Calibri default font)	✓ Fixed (Carlito/Caladea/Liberation) — user-confirmed visually
Arabic text typography (Traditional/Simplified Arabic)	✓ Fixed (KacstOne) — user-confirmed visually

7. Commands reference

Rebuild after a code or Dockerfile change

```
docker compose up -d --build chatbot --no-deps
```

Check font resolution inside the running container

```
docker exec mizan-chatbot fc-match Calibri
docker exec mizan-chatbot fc-match "Traditional Arabic"
docker exec mizan-chatbot fc-list           # full installed font list
docker exec mizan-chatbot fc-list :lang=ar # fonts claiming Arabic coverage
```

If a Dockerfile font/package addition fails with `E: Unable to locate package`, check the real name against the same base image before retrying:

```
docker run --rm python:3.11-slim bash -c \  
"apt-get update -qq && apt-cache search <keyword>; apt-cache policy <candidate-name>"
```

Test via curl

```
TOKEN=$(curl -s -X POST http://localhost:8000/api/auth/login \  
-H "Content-Type: application/json" \  
-d '{"email":"test@example.com","password":"TestPass123!"}' \  
| python -c "import json,sys; print(json.load(sys.stdin)['jwt_token'])")  
  
curl -s -X POST http://localhost:8000/api/convert \  
-H "Authorization: Bearer $TOKEN" \  
-F "file=@sample.docx" \  
-o output.pdf
```

Test via Swagger

1. `http://localhost:8000/docs`
2. `POST /api/auth/login` → copy `jwt_token` → Authorize (padlock, no `Bearer` prefix)
3. `POST /api/convert` → Try it out → choose a `.docx` / `.xlsx` file → Execute → Swagger offers the response as a downloadable PDF directly

Concurrency test (Python, not curl — needed to fire truly simultaneous requests)

```
import concurrent.futures, requests  
  
def convert_once(i):  
    with open("sample.docx", "rb") as f:  
        r = requests.post("http://localhost:8000/api/convert",  
                           headers={"Authorization": f"Bearer {TOKEN}"},  
                           files={"file": (f"test_{i}.docx", f)}, timeout=60)  
        return i, r.status_code, len(r.content)  
  
with concurrent.futures.ThreadPoolExecutor(max_workers=5) as ex:  
    results = list(ex.map(convert_once, range(5)))
```

Generated 2026-07-24 — Mizan.ai, Format Conversion (Feature H) build, testing, and font fixes.